

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Криптографические методы обеспечения информационной безопасности»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

«Модель протокола защищенного соединения»

Выполнили:

Лим Алина Олеговна, студент группы N3349



Проверил:

Таранов Сергей Владимирович, доцент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение	3
1 Ход работы	4
1.1 Визуализация 1 раунда алгоритма хеширования MD5	4
1.1.1 Хеширование текста	4
1.1.2 Структура алгоритма	4
1.1.3 Процесс хеширования	6
1.2 Алгоритм для генерации кодов аутентификации сообщений	12
1.3 Возможности хеширования файлов и сообщений	14
1.4 Генерация Hash based MAC	15
1.4.1 Генерация ключевой пары RSA	15
1.4.2 Шифрование, дешифрование ключа AES	17
1.4.3 Электронная цифровая подпись	18
1.4.4 Сравнение HMAC	18
Заключение	20

ВВЕДЕНИЕ

Цель: изучить подходы к применению криптопримитивов в рамках протоколов для защищенных соединений.

Для достижения поставленной цели необходимо решить следующие задачи:

- выполнить визуализацию 1 раунда алгоритма хэширования SHA-3;
- отразить в отчете алгоритм для генерации кодов аутентификации сообщений;
- протестировать возможности хэширования файлов и сообщений с помощью openssl dgst;
- сгенерировать Hash based MAC.

1 ХОД РАБОТЫ

1.1 Визуализация 1 раунда алгоритма хэширования SHA-3

1.1.1 Хеширование текста

Захешируем с помощью SHA-3 следующий текст:

«A forest is an ecosystem represented by a variety of trees. Forests are deciduous, coniferous, oak and mixed species. Tundra and taiga also belong to forest varieties. The forest is called "the lungs of the planet", because it produces a huge amount of oxygen, and conifers saturate the air with vital nitrogen. But unfortunately, excessive deforestation and forest fires lead to a decrease in oxygen production, as well as to soil erosion and a reduction in the range of animals and insect»

Результат представлен на рисунке 1.

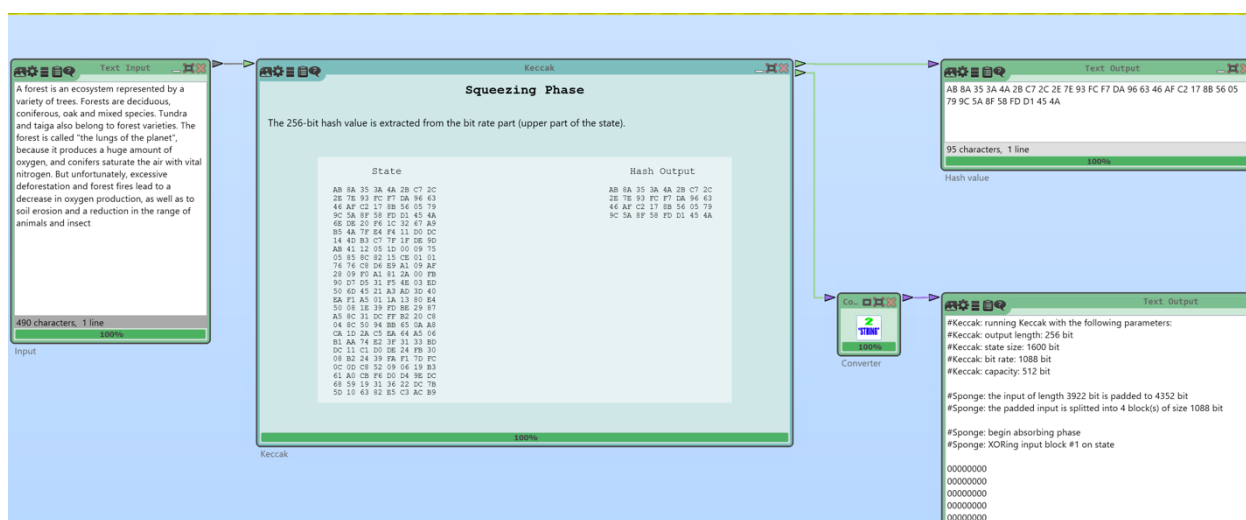


Рисунок 1 – Хеширование текста

1.1.2 Структура алгоритма

SHA-3 (Кессак) – алгоритм хеширования переменной разрядности. Кессак основан на конструкции Sponge (Губка), которая является новым способом проектирования хеш-функций, взамен стратегии итерационного метода Меркла–Дамгора, реализуемой в MD(x).

Губка – это итеративная конструкция для создания функции с произвольной длиной на входе и произвольной длиной на выходе на основе преобразований перестановки.

r и c – части входной строки, сумма которых должны быть равна функции Кессак (рисунок 2). Рекомендуется использовать Кессак – $f\{1600\}$, следовательно $r+c=1600$.

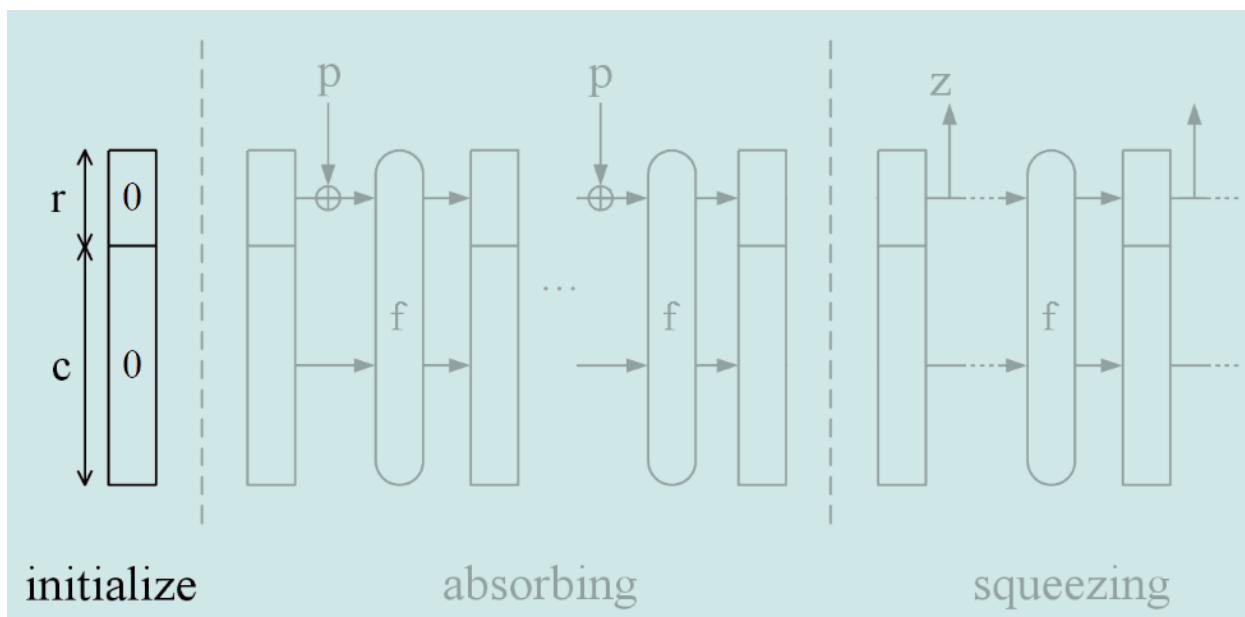


Рисунок 2 – r и c

Алгоритм состоит из 2 этапов – впитывание (absorbing) и отжимание (squeezing). На этапе впитывания входящее сообщение подвергается многораундовым перестановкам f , на этапе отжимания получившееся значение выводится.

Количество функций перестановок вычисляется по формуле $nr=12+2 \cdot l$.

$b=25 \cdot (2^l)$, где b – значение выбранной функции, в нашем случае $b=1600$, следовательно $l=6$.

Значит, $nr=24$, производится 24 раунда перестановки (рисунок 3).

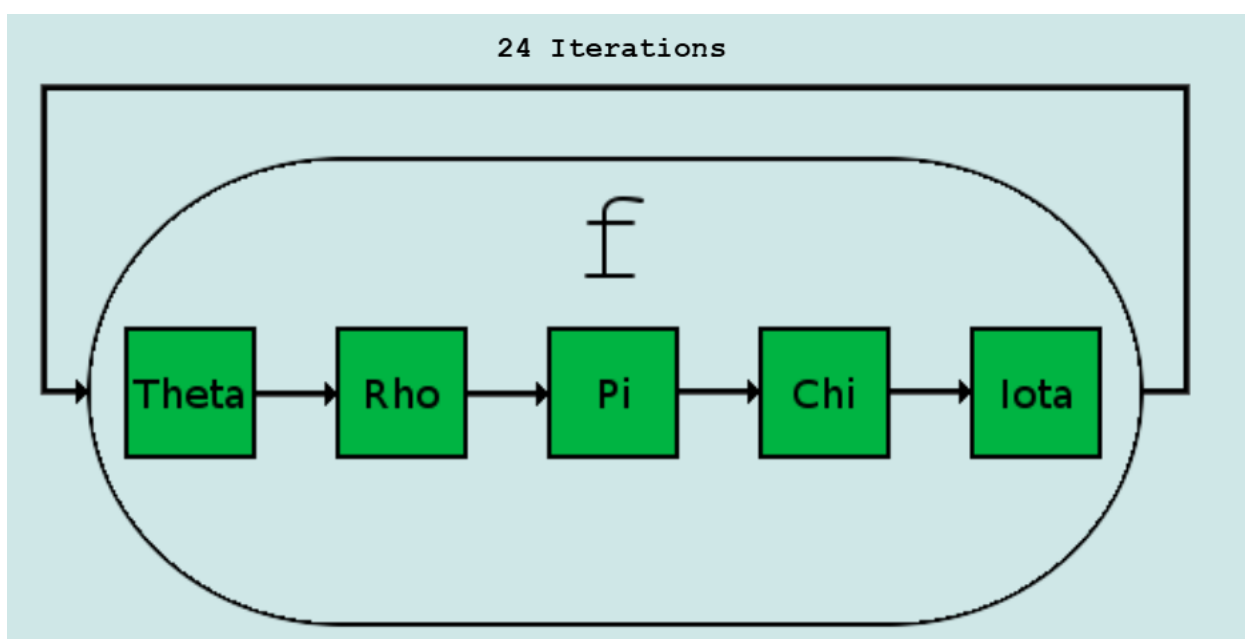


Рисунок 3 – Число перестановок

Для простоты изучения шагов алгоритма процесс хэширования представлен в виде трехмерного куба (рисунок 4). Размер строки (row) и столбца (column) равен 5 битам. Размер полосы (lane) зависит от размера состояния (в нашем случае 64 бита).

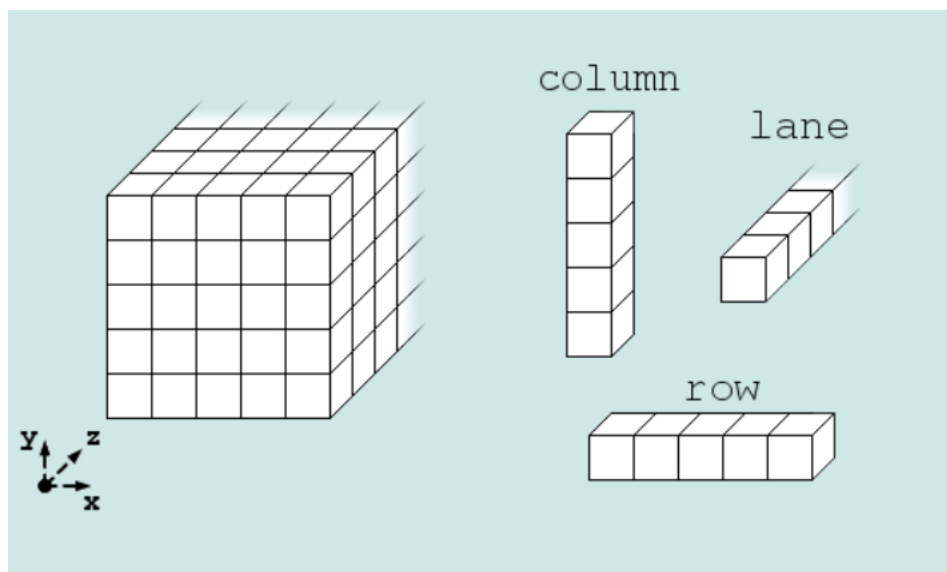


Рисунок 4 – Трехмерный куб

1.1.3 Процесс хеширования

Параметры губки и входная информация представлена на рисунке 5.

Sponge Construction Parameter
State size: 1600 bit
Capacity size: 512 bit
Bit rate size: 1088 bit
Input Information
Input length: 3922 bit = 490 byte
Padded input length: 4352 bit = 544 byte
Number of input blocks (padded input length / bit rate size): 4

Рисунок 5 – Параметры губки и входная информация

1.1.3.1 Фаза впитывания

Ко входному блоку и строке состояния применяется операция XOR (рисунок 6).

Old State	Block #1/4	New State
00 00 00 00 00 00 00 00	41 20 66 6F 72 65 73 74	41 20 66 6F 72 65 73 74
00 00 00 00 00 00 00 00	20 69 73 20 61 6E 20 65	20 69 73 20 61 6E 20 65
00 00 00 00 00 00 00 00	63 6F 73 79 73 74 65 6D	63 6F 73 79 73 74 65 6D
00 00 00 00 00 00 00 00	20 72 65 70 72 65 73 65	20 72 65 70 72 65 73 65
00 00 00 00 00 00 00 00	6E 74 65 64 20 62 79 20	6E 74 65 64 20 62 79 20
00 00 00 00 00 00 00 00	61 20 76 61 72 69 65 74	61 20 76 61 72 69 65 74
00 00 00 00 00 00 00 00	79 20 6F 66 20 74 72 65	79 20 6F 66 20 74 72 65
00 00 00 00 00 00 00 00	65 73 2E 20 46 6F 72 65	65 73 2E 20 46 6F 72 65
00 00 00 00 00 00 00 00	73 74 73 20 61 72 65 20	73 74 73 20 61 72 65 20
00 00 00 00 00 00 00 00	64 65 63 69 64 75 6F 75	64 65 63 69 64 75 6F 75
00 00 00 00 00 00 00 00	73 2C 20 63 6F 6E 69 66	73 2C 20 63 6F 6E 69 66
00 00 00 00 00 00 00 00	65 72 6F 75 73 2C 20 6F	65 72 6F 75 73 2C 20 6F
00 00 00 00 00 00 00 00	61 6B 20 61 6E 64 20 6D	61 6B 20 61 6E 64 20 6D
00 00 00 00 00 00 00 00	69 78 65 64 20 73 70 65	69 78 65 64 20 73 70 65
00 00 00 00 00 00 00 00	63 69 65 73 2E 20 54 75	63 69 65 73 2E 20 54 75
00 00 00 00 00 00 00 00	6E 64 72 61 20 61 6E 64	6E 64 72 61 20 61 6E 64
00 00 00 00 00 00 00 00	20 74 61 69 67 61 20 61	20 74 61 69 67 61 20 61
00 00 00 00 00 00 00 00		00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00		00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00		00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00		00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00		00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00		00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00		00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00		00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00		00 00 00 00 00 00 00 00

Рисунок 6 – Фаза впитывания

1.1.3.2 Theta

Далее строка состояния представлена в виде трехмерной матрицы (куба).

Перебирается каждый столбец куба. Выполняется XOR паритета (равенство или симметрия) двух соседних столбцов (бирюзовый и светло-зеленый). Выводится результат XOR с каждым битом рассматриваемого столбца (зеленый).

Фаза представлена на рисунке 7.

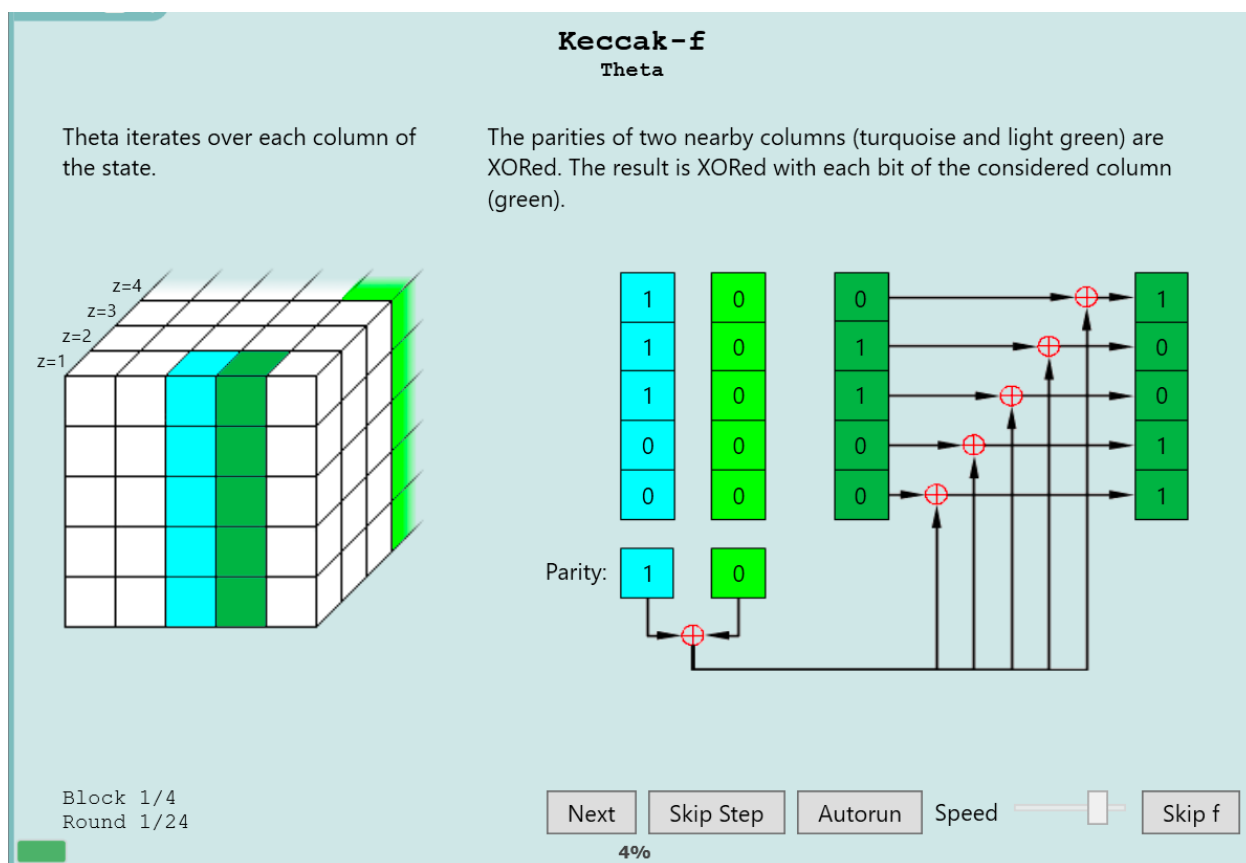


Рисунок 7 – Theta

1.1.3.3 Rho

Перебирается каждая полоса куба. Каждая полоса поворачивается на определенное значение (rotation offset). Верхний зеленый блок представляет полосу до поворота, нижний зеленый блок – после поворота.

Фаза представлена на рисунке 8.

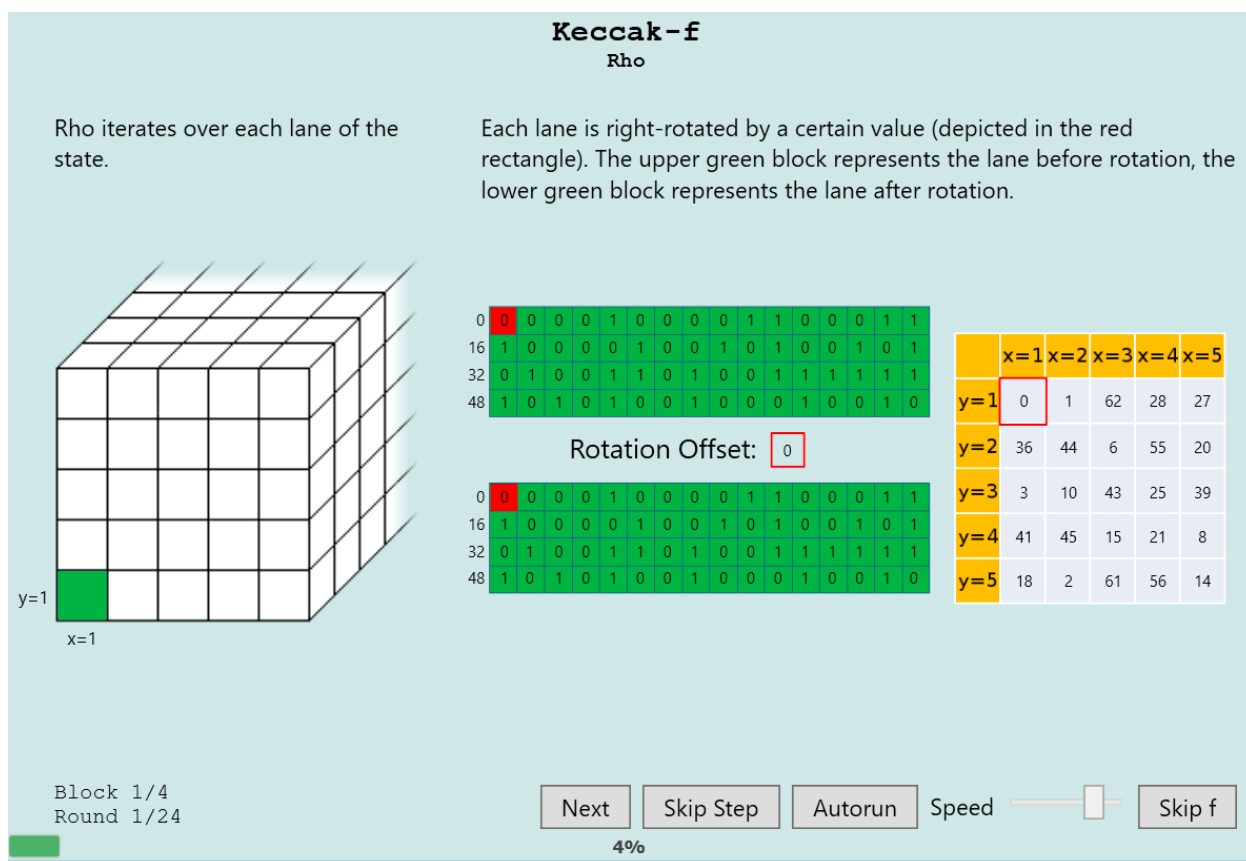


Рисунок 8 – Rho

1.1.3.4 Pi

Меняется расположение полос. Координаты линии куба сдвигаются для улучшения визуализации. Все полосы кроме черной ($x=1$, $y=1$) перемещаются. Правый куб представляет новые позиции линий левого куба. Перемещенные линии выделены серым цветом.

Фаза представлена на рисунке 9.

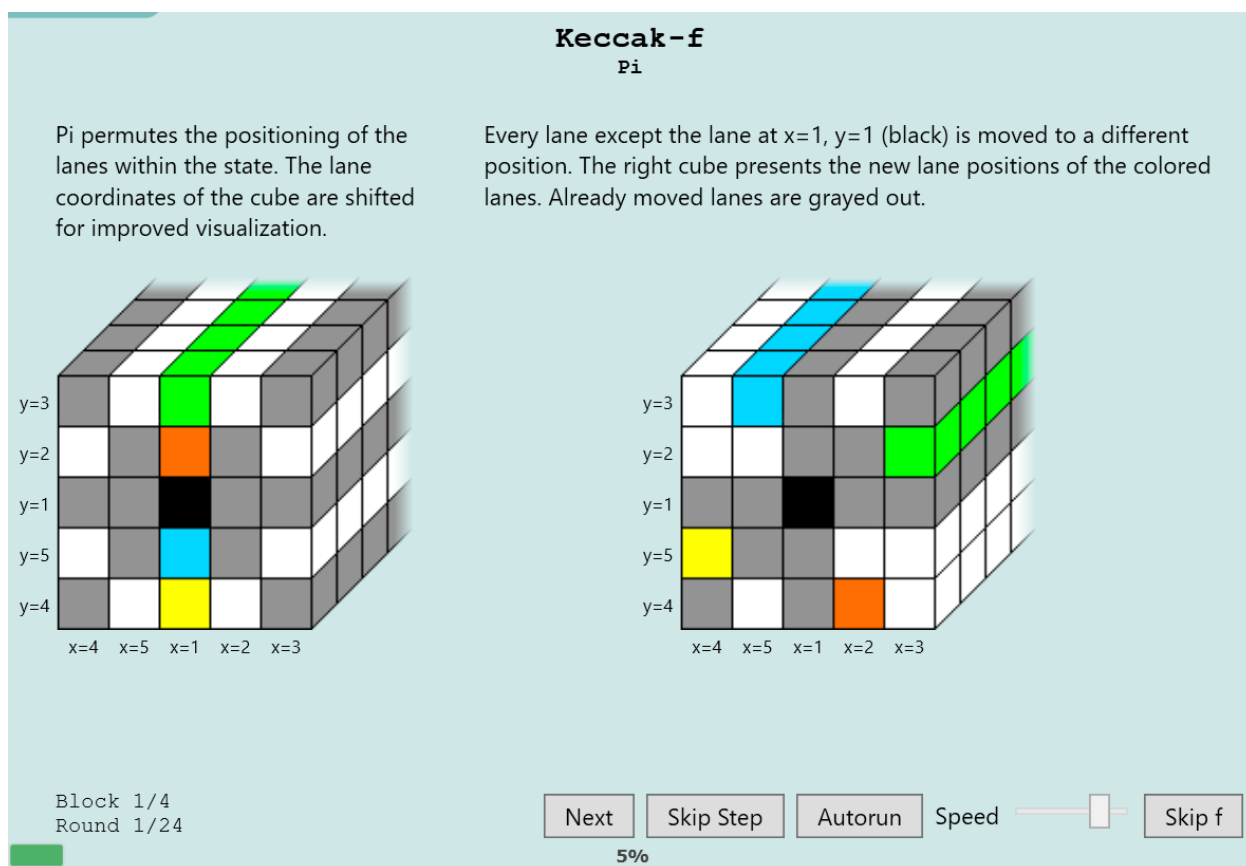


Рисунок 9 – Pi

1.1.3.5 Chi

Перебирается каждая строка куба. Каждый бит строки XOR с конъюнкцией двух битов справа от рассматриваемого. Первый из двух битов инвертируется до конъюнкции.

Фаза представлена на рисунке 10.

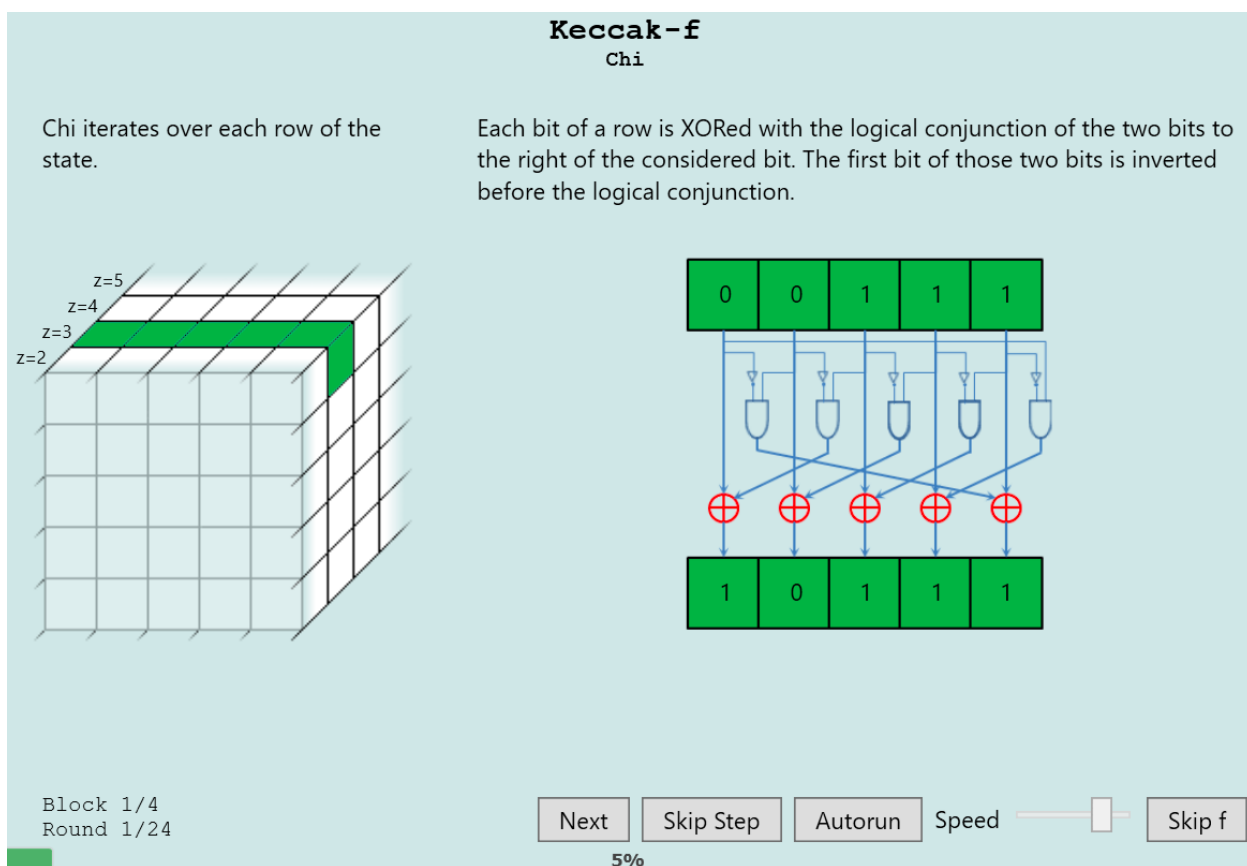


Рисунок 10 – Chi

1.1.3.6 Iota

Выполняется XOR раундовой константы с первой линией ($x=1, y=1$). Значение текущей раундовой константы находится между зелеными блоками. Зеленые блоки представляют старое и новое значения полосы.

Фаза представлена на рисунке 11.

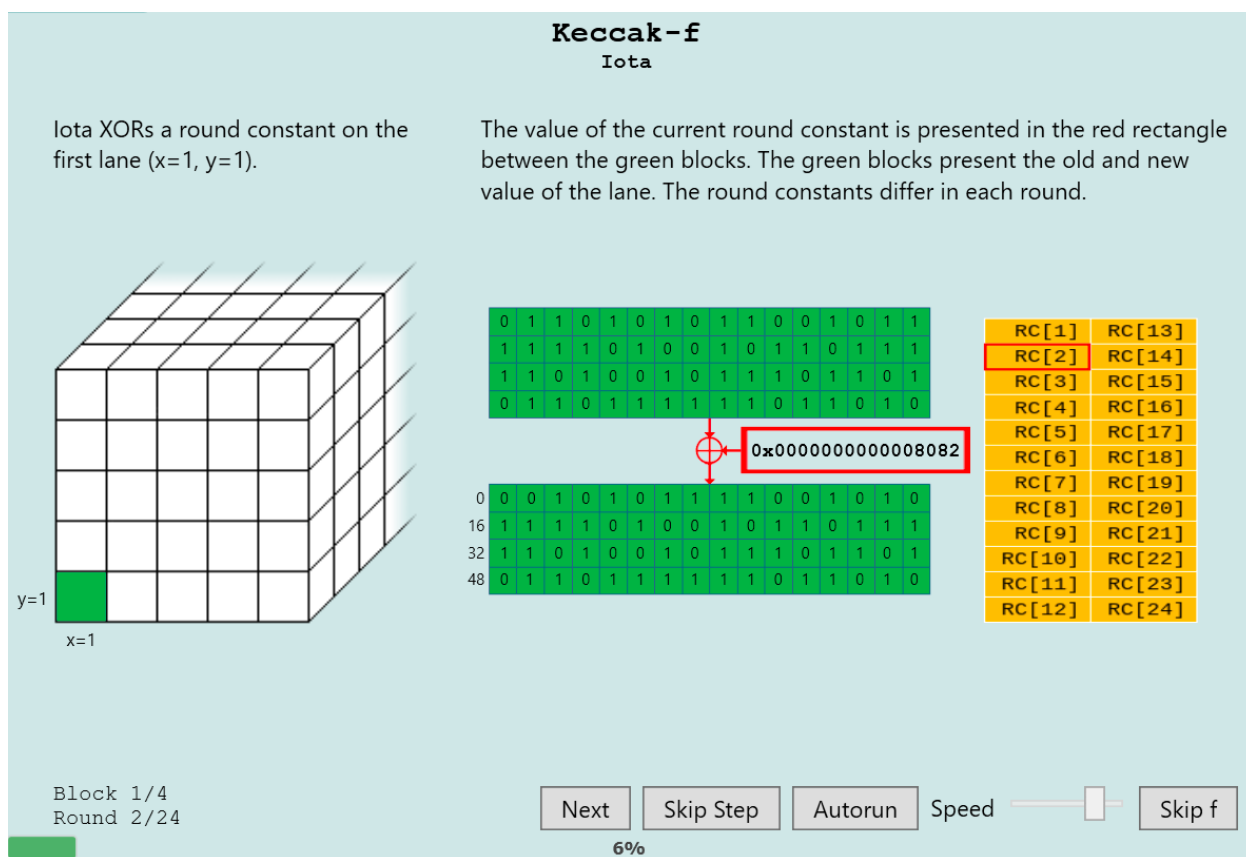


Рисунок 11 – Iota

1.2 Алгоритм для генерации кодов аутентификации сообщений

НМАС – механизм проверки целостности информации, позволяющий гарантировать то, что данные, передаваемые или хранящиеся в ненадежной среде, не были изменены посторонними лицами. Механизм НМАС использует имитовставку (MAC).

MAC – стандарт, описывающий способ обмена данными и способ проверки целостности передаваемых данных с использованием секретного ключа. Два клиента, использующие MAC, как правило, используют общий секретный ключ. НМАС — надстройка над MAC; механизм обмена данными с использованием секретного ключа (как в MAC) и хеш-функций. В названии может уточняться используемая хеш-функция: НМАС-MD5, НМАС-SHA1, НМАС-RIPMD128, НМАС-RIPMD160 и т. п.

Реализация НМАС является обязательной для протокола IPsec. НМАС используется и в других протоколах интернета, например, TLS.

Шаблон НМАС в Cryptool 2 представлен на рисунке 12.

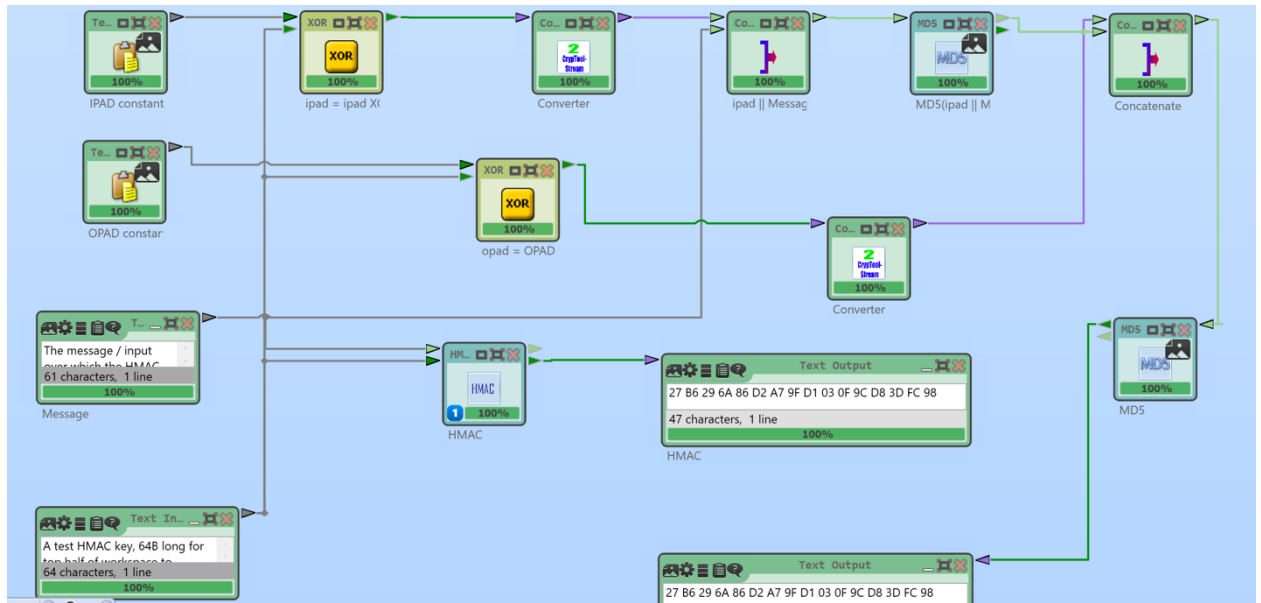


Рисунок 12 – Шаблон HMAC в Cryptool 2

Представленные блоки:

- b , $block_size$ – размер блока в байтах;
- H , $hash$ – хеш-функция;
- $ipad$ – блок вида $(0x36\ 0x36\ 0x36\ \dots\ 0x36)$, где байт $0x36$ повторяется b раз, $0x36$ – константа, магическое число приведенное в RFC 2104, «i» от «inner»;
- K , key – секретный ключ (общий для отправителя и получателя);
- K_0 – изменённый ключ K (уменьшенный или увеличенный до размера блока (до b байт));
- L – размер в байтах строки, возвращаемой хеш-функцией H , L зависит от выбранной хеш-функции и обычно меньше размера блока;
- $opad$ – блок вида $(0x5c\ 0x5c\ 0x5c\ \dots\ 0x5c)$, где байт $0x5c$ повторяется b раз, $0x5c$ – константа, магическое число, приведенное в RFC 2104, «o» от «outer»;
- $text$ – сообщение (данные), которое будет передаваться отправителем и подлинность которого будет проверяться получателем;
- n – длина сообщения $text$ в битах.

Алгоритм HMAC можно записать в виде одной формулы (рисунок 13).

$$HMAC_K(text) = H \left((K_0 \oplus opad) \parallel H \left((K_0 \oplus ipad) \parallel text \right) \right),$$

Рисунок 13 – Основная формула HMAC

В формуле « $\oplus$ » – исключающее «ИЛИ», « $\parallel$ » – склейка строк.

Ниже представлена реализация HMAC-SHA1 (рисунок 14).

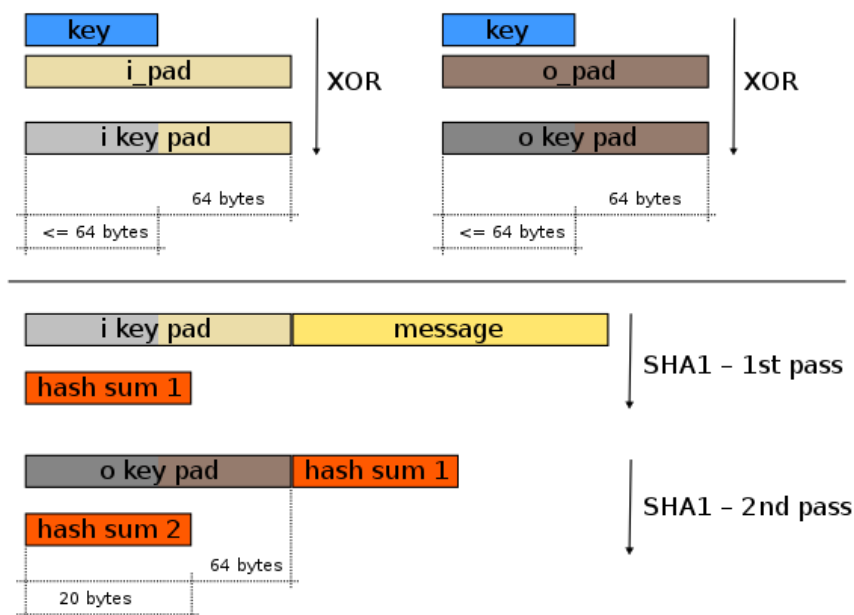


Рисунок 14 – Реализация HMAC-SHA1

1.3 Возможности хэширования файлов и сообщений

Выполним хэширование сообщения. Протестируем следующие алгоритмы: MD5 (рисунок 15), SHA256 (рисунок 16), SHA512 (рисунок 17). Для каждого из алгоритмов захэшируем сообщение повторно, изменив в нём один символ.

```
C:\Users\alinalimmm>echo i will definitely pass this lab! | openssl dgst -md5
MD5(stdin)= 4b7b1fdc67872707ea3a1d6a9805aef4

C:\Users\alinalimmm>echo i will definitely pass this lab? | openssl dgst -md5
MD5(stdin)= 03f0579b1e0ddcced1e8813bd3637530

C:\Users\alinalimmm>echo i will definitely pass this lal? | openssl dgst -md5
MD5(stdin)= 2cef19b1190b97ee2770482ab658906e
```

Рисунок 15 – MD5

```
C:\Users\alinalimmm>echo i will definitely pass this lab! | openssl dgst -sha256
SHA2-256(stdin)= 361c969ab8ad13ab1b655607fab972347fc5e0e82b23f6a2b64558acfe1048f9

C:\Users\alinalimmm>echo i will definitely pass this lab? | openssl dgst -sha256
SHA2-256(stdin)= 3817d290181fd33b104cf40633a62b2ec20446895c94cb6969684d01d92b9868

C:\Users\alinalimmm>echo i will definitely pass this lal? | openssl dgst -sha256
SHA2-256(stdin)= 8a2543bfaf6f2272e2ed55b7c6f1ddf52a4c0378d7fd13dbf19ac1b9ac9fbe8a
```

Рисунок 16 – SHA256

```

C:\Users\alinalimmm>echo i will definitely pass this lab! | openssl dgst -sha512
SHA2-512(stdin)= 2a44759fb35ebecd1b632bafec4c4d3280307cbbde5e81b7d62e6e943f9ff930b48d86ba76591b3b665cc13649830c29ac59277
7049c3cb45c7ec90a47cda237e

C:\Users\alinalimmm>echo i will definitely pass this lab? | openssl dgst -sha512
SHA2-512(stdin)= e8dd2fc1018ffecc692b527b4bf6c3bb26b6cbe2bd589113235e1c150238544b1b24e96107890956925c298ece7c56d9b80f13
126a3d1bafd705069e3a809616

C:\Users\alinalimmm>echo i will definitely pass this lab! | openssl dgst -sha512
SHA2-512(stdin)= 785de1f6f5972bceeca0db66060281a4e7bbaf4c7f416ac195a5f413f41ab7e9e65f77e60cc86ecc52f39ac9f4343bc55bb5c1
6bbca4c80d2035bdc9bd1bd1f4

```

Рисунок 17 – SHA512

Как можно заметить, изменение даже одного символа в сообщении меняет практически весь хэш. Длина хэша полностью зависит от используемого алгоритма (MD5 – 128 бит, SHA256 – 256 бит, SHA512 – 512 бит).

1.4 Генерация Hash based MAC

Сгенерируем HMAC с ключом HMAC-MD5 (рисунок 18). Воспользуемся ключами разной длины.

```

C:\Users\alinalimmm>openssl dgst -hmac key -md5 text.txt
HMAC-MD5(text.txt)= 63530468a04e386459855da0063b6596

C:\Users\alinalimmm>openssl dgst -hmac keeeeeeeeeeeeeeeeeeeeeey -md5 text.txt
HMAC-MD5(text.txt)= f7503bcbdd2c9ad2273389275efd9930

```

Рисунок 18 – HMAC-MD5

Видим, что длина ключа не имеет значения. Заданный ключ дополняется нулями справа, если его длины недостаточно, или хэшируется до тех пор, пока его длина не станет равной размеру блока.

Сгенерируем HMAC с ключами HMAC-SHA256 и HMAC-SHA512 (рисунок 19).

```

C:\Users\alinalimmm>openssl dgst -hmac keeeeeeeeeeeeeeeeeeeeeey -sha256 text.txt
HMAC-SHA2-256(text.txt)= 6aca0c8f74ac21b28aad2a57cb7a25f754d58cef31e4af7813d26aa55e142951

C:\Users\alinalimmm>openssl dgst -hmac keeeeeeeeeeeeeeeeeeeeeey -sha512 text.txt
HMAC-SHA2-512(text.txt)= 71c26b63796c058f585892728e2a3c2341cc9bd62f80ef2916d3bfe0e419fb8238f
9d8a82abddf2f21b73f5e27b808aead2a61f36840941bfc53cb2d43064ed

```

Рисунок 19 – HMAC-SHA256 и HMAC-SHA512

1.4.1 Генерация ключевой пары RSA

Сгенерируем закрытый ключ в файл private.key (рисунок 20).


```

C:\Users\alinalimmm>openssl genrsa -out key.txt

C:\Users\alinalimmm>type key.txt
-----BEGIN PRIVATE KEY-----
MIIEvgIBADANBgkqhkiG9w0BAQEFAASCBAgEAAoIBAQDrPRjROS0YSSDm
Fh3bwjdh1sM4ul/2WrVYMC9apdi7PMQU/UGs9NJnKAxklusfUxjYhxyUtk2sBMX
UgJCZWzjC+5VXlNTHjq60Vm0yS6eps7TamCI9uwnJfmmwKppR2RuG4q1xjd0sNNJ
CeGmTp06TkUPcePYuWrRc1cy/+Vr9j5laSkaBjrkGkZ7AvA5nq3MbeQbXtk2fwRA
ECz024NNpORg6411svV2ySHHypq6z7UTEmwqV+gTy4DRev0hwj5xtuDjQT7YJSPj
xDj+ZTMx4LyoN/q5e1Bs01hXnY7IMqDUWNhNcA5EIrDKVPqVEbsHgGh26KhzIW2V
HGuk011LAGMBAAECggEACDLkG2AG8n/oqHb13xrOWlfItxXxLp4WZaFMTThw4kkU
ZneLOJ18eyxEqPmE7rMP5yASVmYNJwr3p1+W2jyRu89tz5u020ib+xM9MVVfuBeF
QY4aw5CjmzMUu5ipzSs3/18bXwmDt6I8baEYCcVXVTG8K3Uj0qmqZOWZtRqBIwRB
ezY+wAtY64tMYquGo3ON00BF9dQ6yyTrnMzZnm7Jz4I7d2B4q2JD+3SJ5hWh9SmW
GdEbx+qItAK6T/+0Oo8WbTNSkVhahqJeyJ/o6Wd3r0KuasMDRAA0Wmj+VxyRennN
Cr7VCbexfwuZCic2PqlavrxEzjnIPeslhu9VN9AzzQKBgQD+yxrq87nulYexowfD
tgZywiz+nULU0fLUkaFP7HAMIsQ5Puds/+hmd6zqLm+JxRKJ8TD9+exPNyAczr34
uiGDns4Enf7dnHYkCxrWqTfTenCrwVuIOz1lHhQ2GzBzlg/UpThsGng6XjcKNH+3
r4vHkCEXi2BqX4/96FeBACvXDQKBgQDsWkjlVRHM3HFDvukxiV7VnImF8GTx6x1
4fk1/Y1aCI8UFZuXmCZYvbSGp5+KNi15840EwEDKGH0HnbgBG0ACifBgdsSut173
vXYpTpQLtECQnSw0cF3SNMQkhZWq6NgnLzvxYIUJvhXuYQ3yH2sB26QuoB0tIWc+
6akpILZvtwKBgQCpwYhdoaTvYJDXuVci/dOuAdEYkpKZAIhZN+3R4iWWQvOZCf+g
L6AW37rFC8skbzi6zwdlL25SGNg8WUIYxWou109LpDh6ThQoT33CJ1waNN78Srzb
NzbGd/nfUp4lZiWH18zPuZMA0GS7V97/8uWeQFjL5wCF3sWA1Zv3RrXuWQKBgQCU
vg0BmuFbHckPlupPPFQCVtjnmD0+6P5Z7+iCTdtTOeexoYr10E7xshGWXI90z+G2
ycWPGd6sUNT+ogdWyutxrZWVX6lPux8NEjL2s/j/lKS9Xeyf48dnrsVxppAWSwsd
PeCfe3q+Mt5icrnwEk2pkay1mxZBfLAZK7vFokyt6QKBgH+26mr4Sdw9HAEBLKvr
szwQA/tu627uqU6zJ63WsFmdP5jC6C9ls+b0tqEb2Y6kiFT/a1oHkyt4lBFo4Djx
RhuvzmzRGM7j2dTcMCZNgNfk+oF0KnefDjAfnEguIkmqDJFGZlN7CEkx2Ncvdp0Ur
oQoWAeydPPQESgRCRkg6ksYH
-----END PRIVATE KEY-----

```

Рисунок 20 – Генерация ключа

Файл закрытого ключа содержит как закрытый ключ, так и открытый ключ.
Извлечем наш открытый ключ (рисунок 21).


```

C:\Users\alinalimmm>openssl rsa -in key.txt -pubout -out public.txt
writing RSA key

C:\Users\alinalimmm>type public.txt
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAwFNx8dlyein+goBQs7GH
Xmep8D6uG67xaFd0HJ31CrnNbuvu0tjMo2F+FrHYFNmSD4/dTx5/KoiqF4bTf4Tn
63grw9pw09t9iQ72oTs8SeN/7M2/4Dnh0VIxRuVPSIk7UrCkedLOBD9xKsIGwZ4P
9LF1g/7w2XG21XKIjh+D8RK63aa84gH08cE9MrEhT3CxT00k3mbwN9ympQFVcJgj
ZhxZ9bz2/fBe9q9ZNUOusXr00yndV3bSSwUabtHcRBeoN2+GDofl/OgmARt3F0gu
lteap7YZMuIsXcFKcMaQVqePblhD/iXRExmzjVLE3vgb7nu9BSyJc+4/yUGrE4v8
uQIDAQAB
-----END PUBLIC KEY-----

```

Рисунок 21 – Извлечение открытого ключа

1.4.2 Шифрование, дешифрование ключа AES

Сгенерируем ключ AES (рисунок 22).

```

C:\Users\alinalimmm>openssl enc -aes-128-ctr -P
enter AES-128-CTR encryption password:
Verifying - enter AES-128-CTR encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
salt=B3FA87F5C7A1F7A0
key=D61577EAC7D17FAAD22FDA5AC0A5D6A1
iv =A8AABF45441BB72617CD028785629FCB

```

Рисунок 22 – Генерация ключа AES

Поместим данный ключ в текстовый файл aeskey.txt (рисунок 23).

```

C:\Users\alinalimmm>echo D61577EAC7D17FAAD22FDA5AC0A5D6A1 > aeskey.txt

```

Рисунок 23 – Запись ключа в файл

Посмотрим файл aeskey.txt (рисунок 24).

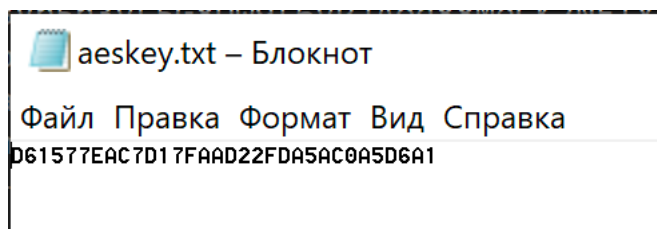


Рисунок 24 – aeskey.txt

Теперь выполним шифрование нашего симметричного ключа при помощи публичного ключа RSA (рисунок 25).

```
C:\Users\alinalimmm>openssl pkeyutl -encrypt -pubin -inkey public.txt -in aeskey.txt -out enc.txt  
C:\Users\alinalimmm>type enc.txt  
аФ!±jђ'±э\≡a┐bR/Л?}±ддбл┐fb◀L♥|т±#
```

Рисунок 25 – Шифрование публичного ключа

Теперь при помощи закрытого ключа расшифруем наш симметричный ключ (рисунок 26).

```
C:\Users\alinalimmm>openssl pkeyutl -decrypt -inkey key.txt -in enc.txt  
D61577EAC7D17FAAD22FDA5AC0A5D6A1
```

Рисунок 26 – Расшифрованный ключ

Как можно заметить, ключ был успешно расшифрован.

1.4.3 Электронная цифровая подпись

Выполним генерацию подписи для зашифрованного сообщения на основе асимметричного алгоритма RSA и проверку подписи RSA. Для этого посчитаем хэш для файла ключа, подпишем файл и проверим подпись (рисунок 27).

```
C:\Users\alinalimmm>type hash.txt  
SHA2-256(enc.txt)= 6eb812992f7d11082a8ffd4c7ca5d0a33ab180214786e6f8d9bcad4798e02649  
C:\Users\alinalimmm>openssl dgst -sha256 -sign key.txt -out sighash.txt hash.txt  
C:\Users\alinalimmm>openssl dgst -verify public.tct -signature sighash.txt hash.txt  
Could not open file or uri for loading private key of public key from public.tct: No such file or  
directory  
C:\Users\alinalimmm>openssl dgst -verify public.txt -signature sighash.txt hash.txt  
Verified OK
```

Рисунок 27 – Генерация хэша и подписи, проверка подписи

1.4.4 Сравнение HMAC

Сгенерируем два HMAC (рисунок 28).

```
C:\Users\alinalimmm>echo i will definitely pass this lab! | openssl dgst -hmac secretkey -md5 -out hmac  
C:\Users\alinalimmm>echo will i?? | openssl dgst -hmac secretkey -md5 -out hmac2  
C:\Users\alinalimmm>type hmac  
MD5(stdin)= 9e263556e0851223651e1cbe2d9f0549  
C:\Users\alinalimmm>type hmac2  
MD5(stdin)= 4249dc02dc4891a5bd6692d77af473b3
```

Рисунок 28 – Генерация двух HMAC

Сравним созданные HMAC при помощи команды FC (рисунок 29).

```
C:\Users\alinalimmm>FC hmac hmac2
Сравнение файлов hmac и HMAC2
***** hmac
MD5(stdin)= 9e263556e0851223651e1cbe2d9f0549
***** HMAC2
MD5(stdin)= 4249dc02dc4891a5bd6692d77af473b3
*****
```

Рисунок 29 – Сравнение разных HMAC

Создадим еще один HMAC для первого сообщения и сравним их (рисунок 30).

```
C:\Users\alinalimmm>echo i will definitely pass this lab! | openssl dgst -hmac secretkey -md5 -out hmac3
C:\Users\alinalimmm>FC hmac hmac3
Сравнение файлов hmac и HMAC3
FC: различия не найдены
```

Рисунок 30 – Сравнение двух одинаковых HMAC

Различия в HMAC найдено не было.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены подходы к применению криптопримитивов в рамках протоколов для защищенных соединений. Приобретенные знания были закреплены на практике с помощью визуализации 1 раунда алгоритма хэширования SHA-3, выполнения алгоритма для генерации кодов аутентификации сообщений, хэширования файлов и сообщений, генерации Hash based MAC, генерации ключевой пары RSA, шифрования и дешифрование ключа AES с помощью ключевой пары RSA, создания и проверки электронной цифровой подписи, сравнения двух различных HMAC. Это позволило получить навыки работы с моделью протокола защищенного соединения. Таким образом, поставленные задачи были выполнены, цель достигнута.